

ALGORITMICKY TĚŽKÉ PROBLÉMY

Mnoho problémů lze v informatice řešit pomocí polynomiálních algoritmů, tzn. běžících v čase $\mathcal{O}(n^k)$ pro nějaké prvné k . Např. problém řazení prvků v poli jsme schopni řešit pomocí různých algoritmů, např. *BubbleSortem*, jehož časová složitost je v nejhorším případě $\mathcal{O}(n^2)$, kde n je počet prvků pole. Podobně hledání nejkratší cesty v grafu dokážeme řešit polynomiálně např. Dijkstrovým algoritmem, který i v případě implementace pomocí pole běží v čase $\mathcal{O}(n^2 + m)$, kde n je počet vrcholů a m počet hran¹. S jistotou lze říci, že polynomiální algoritmy jsou prakticky dobře použitelné.

Bohužel ne vždy je situace takto příznivá. Lze totiž narazit na problémy, na které není známý polynomiální algoritmus a o některých dokonce s jistotou víme, že je v polynomiálním čase řešit nelze. Dobrým příkladem jsou v tomto ohledu např. *Hanojské věže*², kde nejlepší algoritmus je exponenciální, tj. $\mathcal{O}(2^n)$, neboť je vždy potřeba exponenciálně mnoho kroků k vyřešení.

Nás budou zajímat ty nejdůležitější problémy z této oblasti, protože mezi nimi lze nalézt zajímavé vztahy, a posléze si uděláme menší ochutnávku z problematiky P a NP.

1 Problém splnitelnosti

Jako první začneme zdánlivě možná jednoduchým problémem, který však na poli informatiky hraje dosti důležitou roli. Předpokládejme, že máme zadanou nějakou logickou formuli φ o libovolném počtu logických proměnných, označme např. $x_1, x_2, \dots, x_n$. Naší otázkou je, jestli je možné hodnoty proměnných $x_1, x_2, \dots, x_n$ nastavit tak (tj. dosadit hodnoty 0 a 1), aby byla formule splněna³, tj. $\varphi(\dots) = 1$. Takovému ohodnocení budeme říkat *splňující*.

SPLNITELNOST OBECNÉ LOGICKÉ FORMULE

Vstup problému: Logická formule φ .

Výstup problému: 1, pokud je φ splnitelná, jinak 0.

Nejdříve si zopakujeme logické operace a spojky. Mezi ně řadíme $\neg$, $\wedge$, $\vee$, $\implies$, $\iff$.

- **Negace** $\neg x$. *Převrací logickou hodnotu x .*
- **Konjunkce** $a \wedge b$. *Pravdivá, jsou-li obě hodnoty operandů a, b pravdivé.*
- **Disjunkce** $a \vee b$. *Pravdivá, je-li alespoň jedna z hodnot operandů a, b pravdivá.*
- **Implikace** $a \implies b$. *Je-li předpoklad a splněn, je splněn i závěr b . Tj. implikace je nepravdivá, pokud platí a a neplatí b .*
- **Ekvivalence** $a \iff b$. *a platí právě tehdy, když platí b . Tj. ekvivalence je pravdivá, jsou-li hodnoty operandů a, b současně 0 nebo 1.*

Zde konstatujeme, že ve všech příkladech bude mít negace $\neg$ vždy *nejvyšší prioritu* oproti ostatním

¹Může se zdát matoucí, že zde figurují dvě proměnné n a m místo jedné, ale počet hran je omezený (nejvýše mohou být každé dva vrcholy spojeny hranou) a to výrazem $n(n+1)/2$, což je polynom. Tedy všechny prezentované grafové algoritmy běží v polynomiálním čase

²Pro zájemce podrobnější vysvětlení: https://en.wikipedia.org/wiki/Tower_of_Hanoi

³Může se na první pohled zdát, že se jedná o dosti teoretickou záležitost, nicméně později uvidíme, že mnoho jiných problémů má se SAT silnou spojitost.

a	b	$\neg a$	$a \wedge b$	$a \vee b$	$a \implies b$	$a \iff b$
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

operacím. Prioritu ostatních operací budeme stanovovat pomocí závorek.

Příklad 1.1. • $\varphi(x) = x \wedge \neg x \dots$ **Není splnitelná**, pro $x = 0$ i $x = 1$ je $\varphi = 0$.

• $\psi(x) = x \vee \neg x \dots$ **Je splnitelná**, a to jak pro $x = 0$, tak $x = 1$.

• $\chi(a, b, c) = ((a \wedge b) \vee \neg c) \implies (\neg a \wedge \neg c) \dots$ **Je splnitelná**, např. pro $a = 0$, $b = 0$ a $c = 0$.

V příkladu 1.1 výše se jedná o poměrně jednoduché instance, kdy jsme schopni vidět hned, zda je formule splnitelná. Pokud bychom však uvažili nějakou složitější formuli, problém se již značně zkomplikuje.



Obecně pro formuli o n proměnných je třeba prozkoumat řádově 2^n možných ohodnocení. Naivní algoritmus zkoušející všechny možnosti je tak *exponenciální*.

Pro SAT není dodnes známý žádný algoritmus, který by jej uměl řešit pro libovolnou logickou formuli v polynomiálním čase.

1.1 Konjunktivní normální forma

Zkusíme se podívat na formule ve speciálním tvaru, a to tzv. *konjunktivní normální formě*, neboli zkráceně *CNF*.⁴

Každá formule skládá z tzv. **klauzulí** obsahujících tzv. **literály**, což je buď *proměnná*, nebo *její negace*.

- Klauzule jsou ve formuli odděleny *logickou spojkou* $\wedge$
- a v každé klauzuli jsou literály odděleny *logickou spojkou* $\vee$.

$$\varphi(\dots) = \underbrace{(\dots \vee \dots \vee \dots)}_{\text{klauzule}} \wedge (\dots \vee \underbrace{\overbrace{x}^{\text{literál}} \vee \dots}_{\text{literál}}) \wedge (\dots \vee \underbrace{\overbrace{\neg x}^{\text{literál}} \vee \dots}_{\text{literál}}) \wedge (\dots \vee \underbrace{\overbrace{y}^{\text{literál}} \vee \dots}_{\text{literál}}) \wedge \dots$$

Příklad 1.2. • $\varphi(x, y, z) = \neg x \wedge (y \vee z) \dots$ **Je v CNF**. Formule φ obsahuje klauzule x (klauzule o jednom literálu) a $y \vee z$.

• $\psi(a, b) = a \wedge b \dots$ **Je v CNF**.

• $\chi(a, b, c, d) = a \wedge (b \vee (c \wedge d)) \dots$ **Není v CNF**, protože výraz $c \wedge d$ není literál.

Poslední formuli lze však převést do CNF: $a \wedge (b \vee c) \wedge (b \vee d)$.

Z příkladu 1.2 se tak nabízí otázka, zda je možné *libovolnou formuli* převést do CNF. Existuje vůbec pro každou formuli taková ekvivalentní⁵ formule? Ale ano, v logice se i dokazuje, že každé formuli existuje ekvivalentní formule v CNF.

⁴Z angl. *conjunctive normal form*.

⁵Libovolné formule φ a ψ jsou ekvivalentní, pokud pro stejné ohodnocení proměnných platí $\varphi(\dots) = \psi(\dots)$.

Věta 1.3. Pro každou logickou formuli ψ existuje ekvivalentní formule ψ' v CNF.

Toto tvrzení lze, pochopitelně, dokázat, ale zde se tím zabývat nebudeme a zkrátka jej přijmeme jako fakt. Podstatná věc, která z toho plyne, je, že pokud je nějaká formule ψ splnitelná, pak je splnitelná i jí ekvivalentní formule ψ' v CNF a naopak pokud ψ není splnitelná, není splnitelná ani ψ' . Symbolicky

$$\psi \text{ je splnitelná} \iff \psi' \text{ je splnitelná.}$$

1.2 Převod do CNF pomocí tabulky

Podívejme se na způsob, jak lze převést libovolnou logickou formuli do CNF.

Máme obecnou formuli φ s proměnnými $x_1, x_2, \dots, x_n$, pro kterou budeme konstruovat ekvivalentní formuli φ' v CNF. Vycházíme z tabulky pravdivostních hodnot, přičemž pro každé **nepravdivé** ohodnocení vytvoříme *klauzuli* s n literály, kde obecně i -tý literál je

- x_i , pokud v daném ohodnocení je $x_i = 0$,
- jinak $\neg x_i$ (tj. když $x_i = 1$).

Podívejme se na úvod na příklad 1.4 níže.

Příklad 1.4. Máme formuli $\psi(x, y, z) = (x \implies y) \implies z$, pro kterou chceme najít ekvivalentní formuli ψ' v CNF. Nejprve si tedy sestavíme tabulku pravdivostních hodnot. Z tabulky můžeme vidět, že formule

x	y	z	$x \implies y$	$(x \implies y) \implies z$
0	0	0	1	0
0	0	1	1	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

φ je nepravdivá pro

- $x = 0, y = 0, z = 0$,
- $x = 0, y = 1, z = 0$
- a $x = 1, y = 1, z = 0$.

Tedy celkově sestavíme 3 klauzule:

- $(\overbrace{x}^{x=0} \vee \underbrace{y}_{y=0} \vee \overbrace{z}^{z=0})$,
- $(x \vee \underbrace{\neg y}_{y=1} \vee z)$,
- $(\neg x \vee \neg y \vee z)$.

Výsledná formule v CNF bude mít tvar:

$$\psi'(x, y, z) = (x \vee y \vee z) \wedge (x \vee \neg y \vee z) \wedge (\neg x \vee \neg y \vee z).$$

Čtenář se sám může přesvědčit, že formule ψ a ψ' pro libovolné ohodnocení mají stejnou pravdivostní hodnotu.

Nabízí se otázka: „proč tento postup funguje?“ Pokud nějaká formule v CNF není splněna pro určité ohodnocení, pak to nutně znamená, že alespoň jedna z klauzulí není splněna. Každá klauzule však obsahuje literály spojené logickou spojkou $\vee$ a tedy existuje pro ni jen jediné ohodnocení daných literálů, tak, aby nebyla splněna. Z tohoto principu vychází ona konstrukce. Vybereme si ta ohodnocení původní formule, která ji nesplňují a sestrojíme takovou klauzuli, aby právě pro dané ohodnocení nebyla splněna.

Příklad 1.5. $\chi(x_1, x_2, x_3, x_4) = (x_1 \iff x_2) \vee \neg(x_3 \iff x_4)$.

x_1	x_2	x_3	x_4	$x_1 \iff x_2$	$\neg(x_3 \iff x_4)$	$(x_1 \iff x_2) \vee \neg(x_3 \iff x_4)$
0	0	0	0	1	0	1
0	0	0	1	1	1	1
0	0	1	0	1	1	1
0	0	1	1	1	0	1
0	1	0	0	0	0	0
0	1	0	1	0	1	1
0	1	1	0	0	1	1
0	1	1	1	0	0	0
1	0	0	0	0	0	0
1	0	0	1	0	1	1
1	0	1	0	0	1	1
1	0	1	1	0	0	0
1	1	0	0	1	0	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	0	1

$$\chi'(x_1, x_2, x_3, x_4) = (x_1 \vee x_2 \vee \neg x_3 \vee x_4) \wedge (x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_2 \vee \neg x_3 \vee \neg x_4).$$



Problém: Generování tabulky (resp. procházení všech možných ohodnocení) nás přivádí zpět k původnímu problému, a to sice *exponenciální časové složitosti*.

V tomto případě však nejsme schopni postup učinit více efektivní, neboť ve skutečnosti se může stát, že pro obecnou logickou formuli se může jí ekvivalentní formule v CNF až exponenciálně prodloužit.⁶

1.3 Tseitinova transformace

Značně rychlejší způsob převodu formule do CNF předvedl ruský informatik *Grigori Tseitin*. Problém s předchozím postupem v podsekcí 1.2 je ten, že se v nové formuli ψ snažíme ponechat pouze proměnné v původní formuli φ , což způsobí, že formule se může až exponenciálně prodloužit. Postup, který si nyní ukážeme, zavádí do výsledné formule některé proměnné navíc, avšak za cenu rychlejšího algoritmu.

Postup Tseitinovy transformace formule φ lze rozdělit do celkem 4 kroků.

⁶Stačí např. vzít formuli, která je *nesplnitelná*, tj. pro všechna možná ohodnocení platí $\psi(\dots) = 0$. Celkově by tak ekvivalentní formule měla až 2^n klauzulí.

1. Rozdělíme formuli φ na jednotlivé podformule.
2. Pro každou dílčí podformuli φ_i si zavedeme novou proměnnou y_i . Speciálně y_1 je logicky ekvivalentní s celou původní formulí φ , tj. $y_1 \iff \varphi$.
3. Vytvoříme novou formuli ψ následovně:

$$\psi = y_1 \wedge (y_1 \iff \varphi_1) \wedge (y_2 \iff \varphi_2) \wedge \dots \wedge (y_n \iff \varphi_n).$$

Hned se nabízí námitka, že nová formule ψ není v CNF, avšak stačí si uvědomit, že logickou ekvivalenci lze přepsat pomocí $\wedge$ a $\vee$ následovně:

$$a \iff b = (a \implies b) \wedge (b \implies a) = (\neg a \vee b) \wedge (\neg b \vee a). \quad (1)$$

Podformule $\varphi_1, \varphi_2, \dots, \varphi_n$ lze ještě dále přepsat pomocí standardních logických identit. Pro jednoduchost to zde však řešit nebudeme.

Nové proměnné $y_1, y_2, \dots, y_n$ indikují, jaké hodnoty nabývá daný logický výraz v původní formulí pro nějaké konkrétní ohodnocení původních proměnných. Chceme-li tedy zjistit, jaké logické hodnoty nabývá daná formule φ pro určité logické hodnoty jejích proměnných, stačí novým proměnným přiřadit logickou hodnotu podle toho, jaké hodnoty nabývá výraz φ_i , který reprezentují. Přesně proto jsme ostatně konstruovali podformule $y_i \iff \varphi_i$.

Pojďme se podívat na konkrétní příklad.

Příklad 1.6. Máme formuli $\alpha(p, q, r) = ((p \vee q) \wedge r) \implies \neg p$. Chceme pro ni sestavit Tseitinovu transformaci novou formulí β .

Formule α obsahuje tyto dílčí podformule:

- $\neg p$,
- $p \vee q$,
- $(p \vee q) \wedge r$,
- a $((p \vee q) \wedge r) \implies \neg p$ (původní formule α).

Pro ně si zřídíme nové proměnné y_1, y_2, y_3 a y_4 . Tzn. do nové formule β přidáme výrazy

- $y_4 \iff \neg p$,
- $y_3 \iff (p \vee q)$,
- $y_2 \iff (y_3 \wedge r)$
- a $y_1 \iff (y_2 \implies y_4)$.

Výsledná formule bude tedy vypadat následovně:

$$\begin{aligned} \beta(p, q, r, y_1, y_2, y_3, y_4) = & y_1 \wedge (y_1 \iff (y_2 \implies y_4)) \\ & \wedge (y_2 \iff (y_3 \wedge r)) \\ & \wedge (y_3 \iff (p \vee q)) \\ & \wedge (y_4 \iff \neg p). \end{aligned}$$

Ekvivalence ve formulí bychom mohli přepsat pomocí vztahu (1), který jsme si ukazovali výše.

Zkusme nyní formuli vyhodnotit např. pro ohodnocení proměnných $p = 1, q = 0$ a $r = 0$. Původní formule je rovna:

$$\alpha(1, 0, 0) = ((1 \vee 0) \wedge 0) \implies \neg 1 = (1 \wedge 0) \implies \neg 1 = 0 \implies 1 = 1.$$

Hodnoty nových proměnných nastavíme tedy následovně:

- $y_4 = 0$, protože $\neg p = 0$,
- $y_3 = 1$, protože $p \vee q = 1$,
- $y_2 = 0$, protože $y_3 \wedge r = 0$,
- a $y_1 = 1$, protože $y_2 \implies \neg p = 1$.

Nyní můžeme vyhodnotit novou formuli β :

$$\begin{aligned}\beta(1, 0, 0, 1, 0, 1, 0) &= 1 \wedge (1 \iff (0 \implies \neg 1)) \wedge (0 \iff (1 \wedge 0)) \\ &\quad \wedge (1 \iff (1 \vee 0)) \wedge (0 \iff \neg 1) \\ &= 1 \wedge 1 \wedge 1 \wedge 1 \wedge 1 = 1.\end{aligned}$$

Lze se tedy přesvědčit, že formule β má stejnou logickou hodnotu jako původní formule α .

Obecně vždy bude platit, že nově vzniklá formule je splnitelná, právě tehdy když je splnitelná i původní formule. Ovšem podstatnou výhodou oproti postupu s tabulkou je ta, že nová formule je podstatně kratší a lze ji tedy sestrojit mnohem rychleji.



Pro každý z podvýrazů φ_i sestrojíme klauzuli $y_i \iff \varphi_i$, tzn. celkově jich bude lineárně mnoho v počtu podvýrazů.

Při hlubší analýze tohoto postupu (resp. algoritmu) se lze přesvědčit, že velikost výsledné formule ψ je **lineární** ve velikosti původní formule φ .

2 Problém SAT

Již jsme v podsekcí 1.1 nahlédli, že převod do obecné formule do CNF “hrubou silou” je problematický, neboť se nová formule může až exponenciálně prodloužit. V podsekcí 1.3 jsme si proto ukázali Tseitinovu transformaci, která původní postup zefektivnila přidáním nových proměnných do výsledné formule. To znamená, že každou formuli jsme schopni efektivně převést do CNF. Zkusme si tedy původní problém zjednodušit. Předpokládejme nyní, že formule, které máme na vstupu, jsou již v CNF. Tento problém je v informatice velmi význačný a označuje se zkratkou SAT (z angl. *satisfiability*, neboli splnitelnost).

SPLNITELNOST LOGICKÉ FORMULE V CNF (SAT)

Vstup problému: Logická formule φ v CNF.

Výstup problému: 1, pokud je φ splnitelná, jinak 0.

Nyní se již skutečně zaměříme na samotnou splnitelnost dané formule na vstupu. Pro tento účel se obecně využívají tzv. *SAT solvery*, tedy algoritmy, které na vstupu dostanou *formuli* φ v CNF, o níž rozhodnou, zda je či není splnitelná. Přitom využívají formy, v níž je formule zapsaná. Pro tento účel si ukážeme tzn. *algoritmus DPLL* pojmenovaný po svých autorech⁷, kteří s ním přišli roku 1961.

⁷ *Martin (D)avid, Davis (P)utnam, George (L)ogemann a Donald W. (L)oveland.*

2.1 Algoritmus DPLL

Tento algoritmus je založený na rekurzivním postupu. Na vstupu algoritmu je libovolná formule φ v CNF, kterou algoritmus postupně vyhodnocuje. Využívá dvojici procedur – **Unit propagation** a **Pure literal elimination**.

Unit propagation

Tato procedura přijímá hledá v dané formuli φ tzn. *jednotkovou klauzuli*. To je klauzule, která obsahuje **pouze jeden literál**. Např. formule ω níže obsahuje jednotkovou klauzuli.

$$\omega(x, y) = (x \vee \neg y) \wedge \underbrace{\neg y}_{\text{jednotková kl.}} \wedge (\neg x \vee y).$$

Pokud se taková klauzule nachází ve formuli, lze toho využít, neboť pro danou proměnnou existuje vždy jen jediné ohodnocení, takové, aby celá formule byla splněna (každá klauzule musí být splněna). Obecně pokud formule φ obsahuje jednotkovou klauzuli s proměnnou x , pak mohou nastat dva případy. Označme φ' zbytek klauzulí ve formuli φ .

1. Pokud má formule tvar $\varphi = x \wedge \varphi'$, pak nutně $x = 1$.
2. Jinak pokud má formule tvar $\varphi = \neg x \wedge \varphi'$, pak musí být $x = 0$.

Fakt, že jsme schopni hned rozhodnout, jakou ohodnocení musí mít daná proměnná, je v tomto případě hodně ku prospěchu, neboť formuli můžeme zjednodušit. Protože ohodnocení dané jednotkové klauzule již známe, můžeme ji z formule φ odstranit. Zároveň můžeme **odstranit všechny klauzule**, které se vyhodnotí díky proměnné x na 1, neboť nemohou již nijak ovlivnit logickou hodnotu zbylých klauzulí.

Např. pokud formule obsahuje literál x v jednotkové klauzuli, pak hodnota proměnné musí být 1. Tzn. klauzule obsahující x se vyhodnotí též na 1. tj.

$$(\dots \vee x \vee \dots) = 1.$$

Naopak pokud obsahuje literál $\neg x$, pak nutně $x = 0$ a tedy

$$(\dots \vee \neg x \vee \dots) = 1.$$

Dále ještě odstraníme výskyty literálu x , resp. $\neg x$ v *opačné polaritě*, tedy negaci původního literálu. Tzn. pokud literál v jednotkové klauzuli byl x , pak opačná polarita literálu je $\neg x$ a naopak. Důvod, proč jej můžeme odstranit, je ten, že v rámci kterékoliv jiné klauzule se daný literál v opačné polaritě vyhodnotí vždy na 0. Protože je však spojen s ostatními literály pomocí $\vee$, nemůže nijak ovlivnit výslednou logickou hodnotu klauzule a tudíž jej můžeme odstranit. Tedy např. pokud $x = 1$, pak můžeme klauzule tvaru

$$(\dots \vee y \vee \underbrace{\neg x}_{\text{odstraníme}} \vee z \vee \dots)$$

zjednodušit na

$$(\dots \vee y \vee z \vee \dots).$$

Podobně pro $x = 0$:

$$(\dots \vee y \vee x \vee z \vee \dots) = (\dots \vee y \vee z \vee \dots).$$

Příklad 2.1. Máme formuli

$$\chi(x, y, z) = (x \vee y) \wedge y \wedge (z \vee \neg y) \wedge (\neg x \wedge \neg y \vee \neg z).$$

Aplikujeme na ni proceduru *Unit propagation*. Můžeme si všimnout, že klauzule y je jednotková a proměnná y tedy musí být nastavena na 1.

Protože $y = 1$, je zároveň splněna i klauzule $x \vee y$, tj. můžeme ji odstranit. Zároveň můžeme odstranit literál $\neg y$ ze zbylých klauzulí $z \vee \neg y$ a $\neg x \wedge \neg y \vee \neg y$. Celkově dostaneme novou zjednodušenou formuli

$$\chi'(x, y, z) = z \wedge (\neg x \vee \neg z).$$

Pure literal elimination

Ve vstupní formuli φ může nastat situace, kdy se zde určitá proměnná nachází pouze v jedné polaritě. Tzn. pokud se ve formuli nachází proměnná x , pak se v ní nikde nenachází $\neg x$ a naopak pokud se v ní nachází $\neg x$, pak v ní nikde není x . V obou případech můžeme rovnou určit hodnotu dané proměnné, neboť tím splníme všechny klauzule, v nichž se vyskytuje. Ty můžeme odstranit, neboť jsou tím pádem splněny.

$$\dots \wedge \overbrace{(\dots \vee \underbrace{x}_1 \vee \dots)}^1 \wedge \dots \wedge \overbrace{(\dots \vee \underbrace{x}_1 \vee \dots)}^1 \wedge \dots$$

Podobně pro negaci, tj.

$$\dots \wedge \overbrace{(\dots \vee \neg \underbrace{x}_0 \vee \dots)}^1 \wedge \dots \wedge \overbrace{(\dots \vee \neg \underbrace{x}_0 \vee \dots)}^1 \wedge \dots$$

Poté, co jsme si vysvětlili základní dvojici procedur, se nyní nabízí trojice případů, které je ještě třeba prozkoumat.

- *Co když postupným odstraňováním proměnných mi ve formuli vznikne prázdná klauzule?* Taková situace může nastat v případě procedury Unit propagation, kdy odstraňujeme proměnné v opačné polaritě. Pokud nastane situace, že je nějaká klauzule prázdná, pak to znamená, že daná klauzule pod zvoleným ohodnocením **není splnitelná** a tudíž ani celá formule. V takovou chvíli můžeme prohlásit formuli za **nesplnitelnou pod daným ohodnocením**⁸.
- *Co když odstraníme postupně všechny klauzule?* K takové situaci může dojít, pokud jsou všechny klauzule splněny. V takovou určitě chvíli víme, že je původní formule **splnitelná**.
- *Co když po aplikaci obou procedur stále nevíme, zda je φ splnitelná?* V takovou chvíli nám nezbyvá nic jiného, než si náhodně zvolit nějaký literál a prozkoumat obě možnosti jeho ohodnocení. Náhodně si tedy vybereme literál ℓ a zkusíme jej nastavit jednou na hodnotu 1 a podruhé na hodnotu 0. Tím můžeme formuli φ zjednodušit a nově vzniklou formuli φ' rekurzivně aplikovat stejný algoritmus. Pokud lze splnit zbytek formule φ' , pak lze splnit i původní formuli φ .

Toho můžeme dosáhnout tak, že za formuli φ přidáme uměle jednotkovou klauzuli ℓ , resp. v druhém případě $\neg \ell$. Tuto jednotkovou klauzuli si pak opětovně vybere procedura Unit propagation, která vyhodnotí ℓ buď na 1, nebo na 0. Tzn. zavoláme algoritmus DPLL rekurzivně na formule

$$\varphi \wedge \ell \quad \text{a} \quad \varphi \wedge \neg \ell.$$

S těmito informacemi můžeme dát dohromady pseudokód algoritmu.

Zkusme si algoritmus odsimulovat na příkladu.

⁸Je potřeba si uvědomit, že ohodnocení proměnných si během výpočtu volíme, tedy v takovém případě to ještě nutně neznamená, že je formule celkově nesplnitelná. Pouze víme, že dané ohodnocení určitě není splňující

Algoritmus 2.1: DPLL

Vstup: Formule φ v CNF

```
1 while existuje jednotková klauzule ( $\ell$ ) ve formuli  $\varphi$  do
2    $\varphi \leftarrow \text{UnitPropagation}(\ell, \varphi)$ 
3 while existuje literál  $\ell$  v jediné polaritě ve  $\varphi$  do
4    $\varphi \leftarrow \text{PureLiteralElimination}(\ell, \varphi)$ 
   // Všechny klauzule jsou pravdivé
5 if je  $\varphi$  prázdná then return 1;
   // Všechny literály v klauzuli jsou nepravdivé
6 if obsahuje  $\varphi$  prázdnou klauzuli then return 0;
   // Zkusíme ohodnocení pro  $\ell = 0$  a  $\ell = 1$ 
7 Vyber náhodně literál  $\ell$ 
8 return  $\text{DPLL}(\varphi \wedge \ell) \vee \text{DPLL}(\varphi \wedge \neg \ell)$ 
Výstup: 1, je-li formule  $\varphi$  splnitelná, jinak 0.
```

Příklad 2.2. Máme formuli

$$\gamma(x_1, x_2, x_3, x_4) = (x_1 \vee x_3) \wedge \neg x_4 \wedge (\neg x_2 \vee \neg x_3) \wedge x_1 \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_3).$$

Chceme pomocí algoritmu DPLL rozhodnout, zda je splnitelná.

Když se podíváme na pseudokód 2.1 výše, tak první máme aplikovat proceduru *Unit propagation*. Hledáme tedy jednotkové klauzule ve formuli γ , dokud nějaké existují.

- První jednotková klauzule, které si můžeme všimnout, je $\neg x_4$. Hodnota proměnné x_4 tedy nutně musí být 0. Klauzuli tedy odstraníme. Proměnná se nachází ve formuli pouze jednou, tedy daný literál v opačné polaritě nikde odstraňovat nemusíme. Máme nyní formuli

$$\gamma'(x_1, x_2, x_3, x_4) = (x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge x_1 \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_3).$$

- Další jednotková klauzule je x_1 . Proměnná x_1 musí být nutně 1 a klauzuli můžeme odstranit. Dále si však můžeme všimnout, že tím pádem splněna i klauzule $(x_1 \vee x_3)$. Tu můžeme též odstranit. Literál x_1 se dále nachází v opačné polaritě v klauzuli $(\neg x_1 \vee \neg x_3 \vee x_3)$. Literál $\neg x_1$ můžeme také odstranit z formule. Tím se nám formule γ' zjednoduší na

$$\gamma''(x_1, x_2, x_3, x_4) = (\neg x_2 \vee \neg x_3) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_3).$$

Nově vzniklá formule γ'' již žádné další jednotkové klauzule neobsahuje. Dále tedy můžeme aplikovat proceduru *Pure literal elimination*.

- Formule γ'' obsahuje pouze jeden literál v jediné polaritě, a to $\neg x_2$. Tím pádem hodnotu x_2 můžeme nastavit na 0, čímž splníme dvojici klauzulí $(\neg x_2 \vee \neg x_3)$ a $(\neg x_2 \vee x_3)$. Ty můžeme odstranit, čímž dostaneme formuli

$$\gamma'''(x_1, x_2, x_3, x_4) = \neg x_3 \vee x_3.$$

Protože formule γ''' není ani prázdná, ani neobsahuje prázdnou klauzuli, vybereme náhodně jeden literál a prozkoumáme jeho obě možná ohodnocení. V tomto případě nám zbývají pouze literály x_3 a $\neg x_3$. Vyberme např. literál $\neg x_3$. Nyní máme sestavit dvojici nových formulí $(\neg x_3 \vee x_3) \wedge \neg x_3$ a $(\neg x_3 \vee x_3) \wedge x_3$, na které rekurzivně zavoláme DPLL.

- **Formule** $(\neg x_3 \vee x_3) \wedge \neg x_3$. Opět začneme aplikací Unit propagation. K původní formuli γ''' jsme uměle přidali jednotkovou klauzuli $\neg x_3$. Z jí je zjevné, že x_3 musí být 0. Tím zároveň splníme

i klauzuli $(\neg x_3 \vee x_3)$, kterou můžeme odstanit, čímž dostáváme prázdnou formuli. Pure literal elimination aplikovat nemůžeme. Protože je nově vzniklá formule prázdná, vrátíme na výstup 1.

- **Formule** $(\neg x_3 \vee x_3) \wedge x_3$. I zde začneme eliminací jednotkových klauzulí. Tu zde představuje literál x_3 , jehož ohodnocení tedy musí být 1. Tím opět splníme i klauzuli $(\neg x_3 \vee x_3)$, jejímž odstraněním dostaneme zase prázdnou formuli. Tedy opět vrátíme 1.

Obě ohodnocení pro x_3 jsou tedy splňující a tedy i původní formule γ je *splnitelná*.

Ukažme si ještě jeden jednodušší příklad.

Příklad 2.3. Mějme formuli $\tau(x, y, z) = x \wedge (\neg x \vee y \vee z) \wedge \neg x$. Chceme opět pomocí DPLL určit splnitelnost.

Jako první jednotkovou klauzuli můžeme vzít x . Tedy můžeme ji odstranit společně se všemi výskyty daného literálu v opačné polaritě, tj. $\neg x$. Tzn. dostaneme formuli

$$\tau'(x, y, z) = (y \vee z) \wedge ()$$

Všimněte si, že odstraněním literálu $\neg x$ jsme dostali ve formuli prázdnou klauzuli⁹ $()$. Aplikací Pure literal elimination se zbavíme i literálů y a z .

$$\tau''(x, y, z) = ()$$

Formule τ'' obsahuje pouze prázdnou klauzuli. V tuto chvíli algoritmus prohlásí formuli τ za *nesplnitelnou*.

Poslední otázka, která se nabízí, je (jako obvykle) časová složitost algoritmu. Vezmeme-li v potaz nejhorší případ, časová složitost bohužel nevychází úplně příznivě.

Věta 2.4 (Složitost DPLL). Algoritmus DPLL nad formulí φ o n literálech doběhne v čase $\mathcal{O}(2^n)$ a spotřebuje paměť $\mathcal{O}(n)$.

Důkaz. Nyní je třeba se zamyslet, jaká nejhorší situace může při zpracovávání formule φ nastat. Pokud ve formuli neexistuje žádná jednotková klauzule, ani žádný z literálů není pouze v jedné polaritě (tedy procedury Unit propagation a Pure literal elimination nemůžeme vůbec aplikovat), pak musíme náhodně vybrat nějaký z literálů ℓ a rekurzivně zkusit obě možnosti ohodnocení. Pokud toto nastane v každém volání DPLL, vždy eliminujeme pouze jeden literál. Uvažujme, že jedno volání funkce DPLL obecně zabere c kroků, přičemž následně voláme funkci *dvakrát* na formuli o $n - 1$ literálech (jeden jsme eliminovali). Pokud si označíme počet kroků nad formulí o n literálech $T(n)$, pak můžeme dojít k následujícímu vztahu:

$$T(n) = \underbrace{c}_{\text{zpracování } \varphi} + 2 \cdot \overbrace{T(n-1)}^{\text{rekurze}}.$$

Tomuto typu rovnice se v matematice říká tzv. *rekurentní* či *rekurzivní* rovnice¹⁰, neboť hodnota $T(n)$ přímo závisí na předešlé hodnotě $T(n - 1)$. Určitě víme, že prázdnou formuli umíme zpracovat okamžitě, tj. $T(0) = \mathcal{O}(1)$.

⁹Z dané jednotkové klauzule jsme odebrali daný literál v rámci procedury Unit propagation, neprovedli jsme smazání klauzule jako takové.

¹⁰Jedná se vlastně o posloupnost zadanou rekurentně.

Lze ukázat¹¹, že platí $T(n) = \mathcal{O}(2^n)$.

Na formuli o n literálech spotřebuje algoritmus lineární paměť. □

S exponenciální časovou složitostí se sotva spokojíme, avšak v tomto případě je dobré si říci, že zkoumáme složitost v nejhorším případě a v praxi bývají zkoumané logické formule poměrně rychle řešitelné pomocí DPLL.



Problém je však dodnes ten, že na rozhodnutí splnitelnosti logické formule (i těch v CNF) není známý žádný algoritmus, který by běžel v polynomiálním čase, tj. $\mathcal{O}(n^k)$, pro nějaké k . K tomuto tématu se ještě vrátíme v sekci 6.

3 Problém 3-SAT

Zkusme nyní původní problém ještě zjednodušit. Přidáme podmínku, že každá klauzule může obsahovat nejvýše 3 literály. Tím dostáváme variantu problému SAT známý jako *3-SAT*. O formuli, která splňuje zmíněnou formu, budeme říkat, že je v 3-CNF.

SPLNITELNOST LOGICKÉ FORMULE V 3-CNF (3-SAT)

Vstup problému: Formule φ v 3-CNF

Výstup problému: 1, je-li φ splnitelná, jinak 0.

¹¹*Pro zájemce:* U rekurentní rovnice se snažíme najít předpis pro $T(n)$, takový, aby jeho hodnota nezávisela na jiných hodnotách T . Řešení obsahuje práci s geometrickými řadami. Víme, že platí $T(n) = c + 2 \cdot T(n-1)$. Zkusme podle tohoto předpisu rozepsat $T(n-1)$.

$$T(n) = c + 2 \cdot T(n-1) = c + 2 \cdot (c + 2 \cdot T(n-2)) = 3c + 4 \cdot T(n-2)$$

Zkusme rozepsat opět i $T(n-2)$.

$$T(n) = 3c + 4 \cdot T(n-2) = 3c + 4 \cdot (c + 2 \cdot T(n-3)) = 7c + 8 \cdot T(n-3)$$

Obecně lze pororovat, že k rozepsáních dostaneme pro $T(n)$ následující vzorec:

$$T(n) = c \cdot \sum_{i=0}^{k-1} 2^i + 2^k \cdot T(n-k).$$

Je důležité si nyní uvědomit, že jedinou hodnotu, kterou známe, je $T(0) = \mathcal{O}(1)$. Pokud vzorec rozepíšeme n -krát, tj. $k = n$, můžeme se tak zbavit rekurence v rovnici.

$$T(n) = c \cdot \sum_{i=0}^{n-1} 2^i + 2^n \cdot T(0) = c \cdot \sum_{i=0}^{n-1} 2^i + 2^n \cdot \mathcal{O}(1) = c \cdot \sum_{i=0}^{n-1} 2^i + 2^n$$

Řada $\sum_{i=0}^{n-1} 2^i$ je geometrická s kvocientem $q = 2$ a prvním členem rovnému $2^0 = 1$, přičemž celkový počet členů je n . Můžeme na ni aplikovat známý vzorec pro součet.

$$T(n) = c \cdot \frac{2^n - 1}{2 - 1} + 2^n = c \cdot (2^n - 1) + 2^n = (c + 1)2^n - c.$$

Protože nás však zajímá asymptotické chování algoritmu, konstanty $c + 1$ a c můžeme zanedbat. Z toho již můžeme vidět, že složitost je exponenciální.

$$T(n) = \mathcal{O}((c + 1)2^n - c) = \mathcal{O}(2^n)$$

Nabízí se otázka, jestli jsme si skutečně nějak pomohli. Zdánlivě ano, neboť máme kratší klauzule, avšak 3-SAT převdstavuje, překvapivě, stejně těžkou úlohu jako původní problém SAT. Proč? Lze totiž ukázat, že pokud bychom uměli rychle řešit 3-SAT, uměli bychom rychle řešit i původní SAT a naopak. To ukážeme tak, že jeden problém v jistém smyslu “převést” na ten druhý. Pro určitou instanci (tedy formuli) problému SAT můžeme sestavit odpovídající instanci problému 3-SAT, jejímž vyřešením získáme odpověď i na původní problém. Podívejme se na tuto situaci blíže v podsekcí 3.1.

3.1 Převod SAT na 3-SAT

Popíšeme si nyní obecný způsob, jak převést libovolnou formuli v CNF (instance problému SAT) na formuli v 3-CNF (instance problému 3-SAT). Předpokládejme, že máme zadanou nějakou formuli φ v CNF a chceme pro ni sestavit formuli ψ v 3-CNF, takovou, aby byla splnitelná právě tehdy, když je splnitelná i původní formule φ . Pro formuli φ mohou nastat 2 případy:

1. formule obsahuje klauzule délky 3 a kratší,
2. nebo ve formuli existuje klauzule délky $k > 3$.

V prvním případě není co řešit, neboť φ již je v 3-CNF, tedy stačí položit $\psi = \varphi$ a jsme hotovi. V druhém případě musíme dlouhou klauzuli o k literálech rozložit na kratší. Tuto klauzuli¹² rozložíme na dvojici nových klauzulí pomocí nové proměnné x . Označme si literály dané dlouhé klauzule $\ell_1, \ell_2, \dots, \ell_k$. První dvojici ℓ_1 a ℓ_2 si označíme α a zbytek β .

$$\underbrace{\ell_1 \vee \ell_2}_{\alpha} \vee \overbrace{\ell_3 \vee \dots \vee \ell_k}^{\beta} = \alpha \vee \beta$$

Nyní přidáme novou proměnnou x a klauzuli $\alpha \vee \beta$ rozdělíme na dvojici klauzulí následovně:

$$(\alpha \vee x) \wedge (\beta \vee \neg x).$$

Podívejme se nyní na délky nových klauzulí $\alpha \vee x$ a $\beta \vee \neg x$.

- Klauzule $\alpha \vee x = \ell_1 \vee \ell_2 \vee x$ obsahuje 3 literály, takže s ní dále nic provádět nemusíme.
- Klauzule $\beta \vee \neg x = \ell_3 \vee \dots \vee \ell_k \vee \neg x$ obsahuje $k - 1$ literálů. To znamená, že jsme zkrátily původní klauzuli o jeden literál. Pokud $k - 1 > 3$, můžeme postup pro tuto klauzuli opakovat.

Nyní zbývá dořešit, jak to bude s hodnotou proměnné x . Podobně jako u Tseitinovy transformace (viz podsekcí 1.3), i zde bude její hodnota záviset na hodnotách ostatních proměnných, konkrétně výrazu α .

- Pokud $\alpha = 1$, pak nastavíme $x = 0$. Původní klauzule $\ell_1 \vee \dots \vee \ell_k = \alpha \vee \beta$ je splněna díky α . Nová dvojice klauzulí je pak splněna pomocí α a x :

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (1 \vee 0) \wedge (\beta \vee 1) = 1 \wedge 1 = 1.$$

- Pokud $\alpha = 0$, pak nastavíme $x = 1$. Klauzule $\alpha \vee \beta$ musí být splněna díky β . Potom platí:

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (0 \vee 1) \wedge (1 \vee 0) = 1 \wedge 1 = 1.$$

Pokud by však $\beta = 0$, pak by původní klauzule $\alpha \vee \beta$ nebyla splněna a potom

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (0 \vee 1) \wedge (0 \vee 0) = 1 \wedge 0 = 0,$$

tzn. ani nová dvojice klauzulí by nebyla splněna, což chceme.

¹²Pokud φ obsahuje více dlouhých klauzulí, postup opakujeme pro každou z nich zvlášť.

Tím jsme dostali obecný postup pro převod formule z CNF do 3-CNF.

Příklad 3.1. Máme formuli $\varphi(a, b, c, d) = a \vee b \vee \neg c \vee \neg b \vee d \vee \neg d$ o jedné klauzuli, kterou chceme převést do 3-CNF. K tomu si pořídíme novou proměnnou x . Podle postupu popsaného výše si sestavíme dvojici nových klauzulí $\alpha \vee x$ a $\beta \vee \neg x$.

$$\underbrace{a \vee b}_{\alpha} \vee \overbrace{\neg c \vee \neg b \vee d \vee \neg d}^{\beta}$$

Tím dostáváme novou formuli

$$\varphi'(a, b, c, d, x) = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee d \vee \neg d \vee \neg x).$$

První klauzule $a \vee b \vee x$ obsahuje 3 literály, avšak klauzule $\neg c \vee \neg b \vee d \vee \neg d \vee \neg x$ je však stále dlouhá (všimněte si ale, že obsahuje o jeden literál méně než původní klauzule). Postup tedy opakujeme.

Vytvoříme si opět novou proměnnou, označme ji y . Podobně jako výše i zde rozložíme danou klauzuli na dvě kratší.

$$\underbrace{(\neg c \vee \neg b \vee y)}_{\alpha'} \wedge \overbrace{(d \vee \neg d \vee \neg x \vee \neg y)}^{\beta'}$$

Tedy dostáváme opět novou formuli

$$\varphi'(a, b, c, d, x, y) = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee \neg x \vee \neg y).$$

Třetí klauzule je stále ještě dlouhá, tedy postup zopakujeme ještě jednou s novou proměnnou z .

$$\underbrace{(d \vee \neg d \vee z)}_{\alpha''} \wedge \overbrace{(\neg x \vee \neg y \vee \neg z)}^{\beta''}$$

Celkově dotáváme formuli, která je již v 3-CNF:

$$\psi(a, b, c, d, x, y, z) = (a \vee b \vee x) \wedge (\neg c \vee \neg b \vee y) \wedge (d \vee \neg d \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Nyní si zkusíme určit hodnoty pomocných proměnných x, y, z na základě zadaného ohodnocení proměnných v původní formuli φ . Zkusme např. $a = 0, b = 0, c = 0, d = 1$. Lze jednoduše vypočítat, že $\varphi(0, 0, 0, 1) = 1$.

- Protože ve φ' platí, že $a \vee b = 0 \vee 0 = 0$, tak položíme $x = 1$.
- Dále ve φ'' máme $\neg c \vee \neg b = 1 \vee 1 = 1$, tedy $y = 0$
- a jako poslední pro ψ platí $d \vee \neg d = 1 \vee 0 = 1$, tj. opět $z = 0$.

Pomocné proměnné tedy nastavíme následovně: $x = 1, y = 0, z = 0$. Můžeme se přesvědčit, že nová formule ψ je v tomto ohodnocení také splněna:

$$\begin{aligned} \psi(0, 0, 0, 1, 1, 0, 0) &= (0 \vee 0 \vee 1) \wedge (1 \vee 1 \vee 0) \wedge (1 \vee 0 \vee 0) \wedge (0 \vee 1 \vee 1) \\ &= 1 \wedge 1 \wedge 1 \wedge 1 \\ &= 1. \end{aligned}$$

Tímto způsobem jsme ukázali, že problém SAT lze “převést” na 3-SAT. Opačný směr je triviální, protože každá formule v 3-CNF už je i v CNF, takže nic převádět nemusíme.

Zkusme se zamyslet nad rychlostí onoho převodu formule. Obecně formuli o $k > 3$ literálech rozložíme na $k - 2$ nových formulí. Celkově tedy strávíme převodem lineární čas, tj. $\mathcal{O}(n)$, kde n je počet literálů formule.

Z tohoto vlastně vyplývá, že tyto problémy v jistém smyslu “stejně těžké”. Kdybychom uměli vyřešit polynomiálně 3-SAT, pak i SAT bychom uměli vyřešit s nějakým zpomalením. Obecně uvažujme, že nějaká formule φ v CNF obsahuje n literálů. Tu převedeme na novou formuli ψ , která bude obsahovat nejvýše $n - 2$ nových proměnných, tedy její délka bude $n + n - 2 = 2n - 2$, tedy má stále lineární délku. Určení splnitelnosti výsledné formule ψ by tedy zabralo lineární čas $\mathcal{O}(2n - 2) = \mathcal{O}(n)$.

Toto byla ukázka tzv. **polynomiálního převodu** mezi problémy. Pojďme si toto trochu rigorózněji definovat.

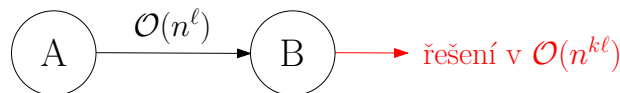
Definice 3.2 (Polynomiální převoditelnost). Řekneme, že libovolný problém A je polynomiálně převoditelný na problém B , píšeme $A \rightarrow B$, pokud každý vstup x problému A lze převést v polynomiálním čase na vstup x' problému B , přičemž $A(x) = B(x')$.

Co tato definice říká? Pokud platí $A \rightarrow B$, pak odpověď na vstup x' je zároveň i odpovědí na původní vstup x . V našem případě je formule v CNF φ splnitelná (to je naše x), právě když je nová formule v 3-CNF ψ (to je x') splnitelná.

Zároveň $A \rightarrow B$ znamená, že problém B je alespoň tak těžký jako problém A .¹³



Pokud pro problém A neexistuje polynomiální algoritmus, pak ani pro B nemůže existovat. Naopak pokud B lze řešit polynomiálně, pak i A lze řešit polynomiálně. Pokud obecně budeme mít pro problém A vstup velikosti n , přičemž převod potrvá $\mathcal{O}(n^\ell)$ pro nějaké ℓ , pak délka nového vstupu bude též $\mathcal{O}(n^\ell)$. Pokud problém B umíme řešit v čase $\mathcal{O}(b^k)$ pro nějaké k , kde b je velikost vstupu, pak pro nový vstup potrvá výpočet $\mathcal{O}(n^{k\ell})$. Celkově tedy strávíme čas $\mathcal{O}(n^\ell)$ převodem a pak čas $\mathcal{O}(n^{k\ell})$ samotným řešením problému B . To celkově potrvá $\mathcal{O}(n^\ell + n^{k\ell})$, což je zase polynom. Viz obrázek 1.



Obrázek 1: Schéma polynomiálního převodu.

4 Problém nezávislé množiny v grafu

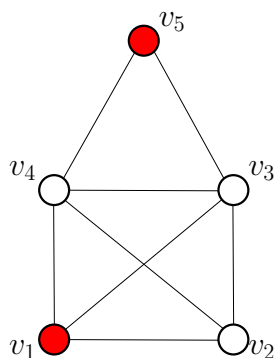
Od problému SAT, resp. 3-SAT se na chvíli přesuneme ke zdánlivě odlišnému problému z teorie grafů. Později si ukážeme i jeho praktický důsledek v podsekcí 4.2. Nejdříve však bude dobré začít definicí.

Definice 4.1 (Nezávislá množina). Mějme libovolný graf $G = (V, E)$. Množina $N \subseteq V$ je *nezávislá*, pokud žádné dva vrcholy nejsou spojeny hranu.

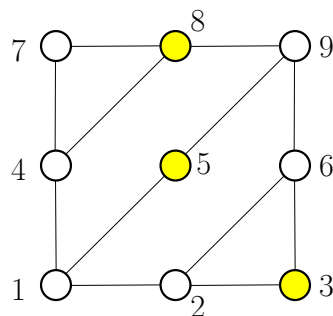
Definice 4.1 jednoduše říká, že jakákoliv vybraná množina vrcholů z daného grafu je nezávislá, pokud mezi všemi vybranými vrcholy nevede hrana. Podívejme se na konkrétní příklad.

Příklad 4.2. Nezávislé množiny M_1 a M_2 v grafech G_1 a G_2 (viz obrázky 2a a 2b).

¹³Mnemotechnická pomůcka: „Obtížnost teče ve směru šipky.“



(a) Nezávislá množina vrcholů $M_1 = \{v_1, v_2\}$ v G_1



(b) Nezávislá množina vrcholů $M_2 = \{3, 5, 8\}$ v G_2

Zde poznamenejme, že na existenci nezávislé množiny nemá smysl se ptát, neboť v libovolném neprázdném grafu (tedy má alespoň jeden vrchol) existuje nezávislá množina, např. libovolná jednovrcholová množina je určitě z definice nezávislá. Zajímavější otázkou je spíše, zda v zadaném grafu existuje nezávislá množina, která má alespoň určitou velikost (tzn. počet vrcholů).

NEZÁVISLÁ MNOŽINA V GRAFU (NzMNA)

Vstup problému: Graf $G = (V, E)$ a číslo $k \in \mathbb{N}$.

Výstup problému: 1, pokud v grafu G existuje nezávislá množina velikosti alespoň k , jinak 0.

Jak později uvidíme, otázka existence nezávislé má i své aplikace při řešení praktických problémů (opět viz podsekcce 4.2), avšak ještě než tak učiníme, zkusme se podívat na souvislosti s problémy, které již známe.

4.1 Převod 3-SAT na NzMna

Mezi dvojicí problémů NzMna a 3-SAT existuje přímá souvislost oběma směry. Problém 3-SAT lze tedy převést na problém nezávislé množiny a naopak. Pro účely tohoto textu si však ukážeme pouze převod jedním směrem, ten druhý je již podstatně složitější¹⁴.

Začínáme s formulí φ , která je ve formě 3-CNF. Uvažujme, že obsahuje obecně k klauzulí, kde každá obsahuje maximálně 3 literály. Chceme pro tuto formuli sestavit graf G a číslo n takové, že v něm existuje nezávislá množina velikosti alespoň n .

$$\varphi = \underbrace{(\dots)}_{\text{max. 3 literály}} \wedge (\dots) \wedge \dots \wedge (\dots)$$

Aby byla formule φ splněná, musí být splněny všechny její klauzule. K tomu je potřeba, aby v každé klauzuli existoval alespoň jeden literál, který ji bude splňovat (může jich být i více). Z každé klauzule tedy vybereme jeden literál, který ji bude splňovat. Je však nutné upozornit, že nesmíme vybírat konfliktně, tedy např. nelze v jedné klauzule vybrat jako splňující literál x a v jiné klauzuli $\neg x$.

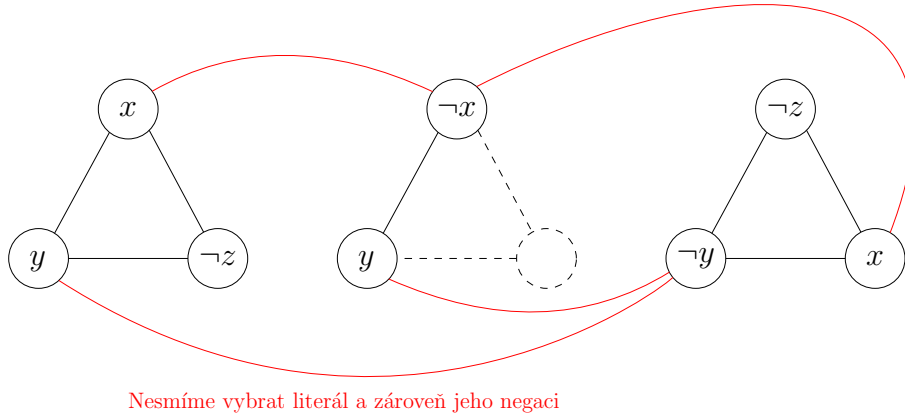
$$(\dots) \wedge (\dots \vee \underbrace{x}_{\text{vybereme}} \vee \dots) \wedge \dots \wedge (\dots \vee \underbrace{\neg x}_{\text{nesmíme vybrat}} \vee \dots) \wedge \dots$$

¹⁴Pro zájemce viz kniha *Průvodce labyrintem algoritmů* (2. vydání), str. 471

Pro každou klauzuli vytvoříme v novém grafu G **trojúhelník**, kde jeho vrcholy představují literály dané klauzule. A jako poslední spojíme hranou **každý literál s jeho negací**. Např. pro formuli

$$\psi(x, y, z) = (x \vee y \vee \neg z) \wedge (\neg x \vee y) \wedge (\neg z \vee \neg y \vee x)$$

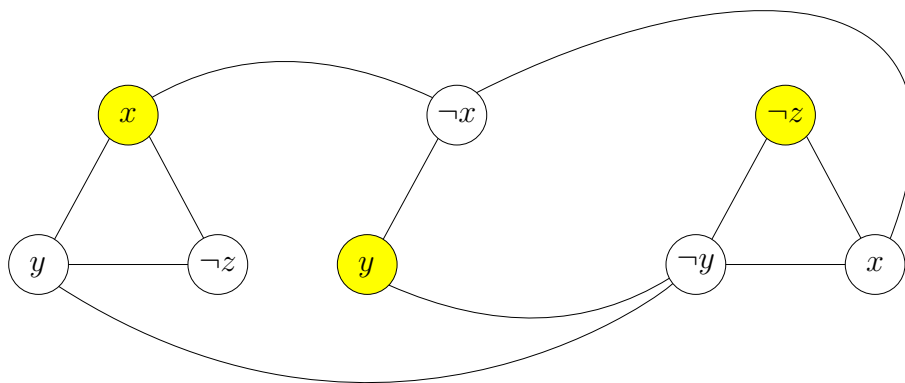
je příslušný graf znázorněný na obrázku 3.



Obrázek 3: Ukázka sestrojeného grafu pro formuli ψ .

Pojďme se nyní promyslet danou konstrukci grafu G . Pokud v nějaké klauzuli vybereme jeden literál, který ji bude splňovat, tak v daném grafu G budou vrcholy příslušné daným literálům tvořit nezávislou množinu. Zároveň však nemůže dojít k situaci, kdy bychom vybrali jednou vybrali literál a v jiné klauzuli jeho negaci, protože příslušné dvojice jsou v grafu G spojeny hranou.

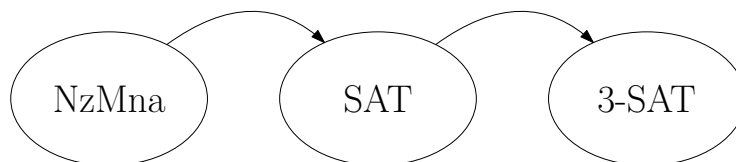
Např. pro již zmíněnou formuli ψ , která je splnitelná např. pomocí ohodnocení $x = 1, y = 1$ a $z = 0$ je příslušná nezávislá množina znázorněna na obrázku 4, kde jsou splňující literály jednotlivých klauzulí x, y a $\neg z$.



Obrázek 4: Ukázka nezávislé množiny příslušné splňujícím literálům pro formuli ψ .

Jako poslední musíme ještě určit číslo n . To je ale velice jednoduché, protože v každé klauzuli vybíráme právě jeden literál a celkově těchto klauzulí je obecně k . Tedy v takto sestrojeném grafu se nachází právě k trojúhelníků, v každém z nich vybíráme právě jeden vrchol, tedy $n = k$.

K převodu NzMna na 3-SAT přidáme jen krátký komentář. Idea převodu je využití klasického SATu. Zadaný graf G a číslo k se převede na formuli v CNF a ta se následně převede do 3-CNF. Problém SAT tak využijeme v jistém smyslu jako “prostředníka” (viz obrázek 5).



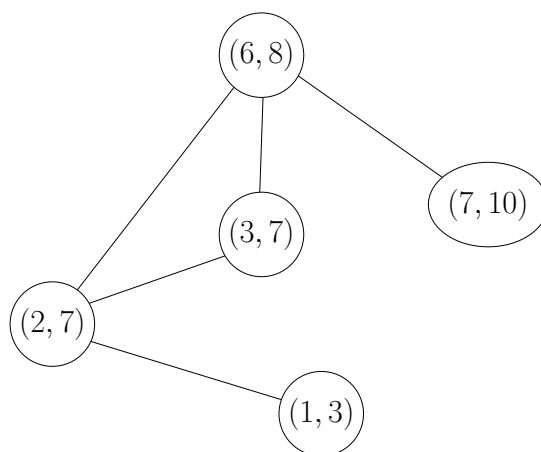
Obrázek 5: Idea převodu NzMna $\rightarrow$ 3-SAT.

4.2 Aplikace NzMna – barvení intervalového grafu

Nyní se podíváme na jednu z aplikací nezávislé množiny v praktických problémech.

Představme si, že máme n vyučovacích hodin ve škole, které chceme rozvrhnout do co nejmenšího počtu učeben tak, aby se žádné dvě z nich nepřekrývaly. Každá vyučovací hodina má čas začátku a čas svého konce. Matematicky toto můžeme reprezentovat pomocí intervalu (a, b) , kde a je čas začátku a b je čas konce. Jednotlivé hodiny tedy můžeme reprezentovat pomocí intervalů $(a_1, b_1), (a_2, b_2), (a_3, b_3), \dots, (a_n, b_n)$. V tomto případě záměrně volíme otevřené intervaly. Později uvidíme, proč se nám toto hodí.

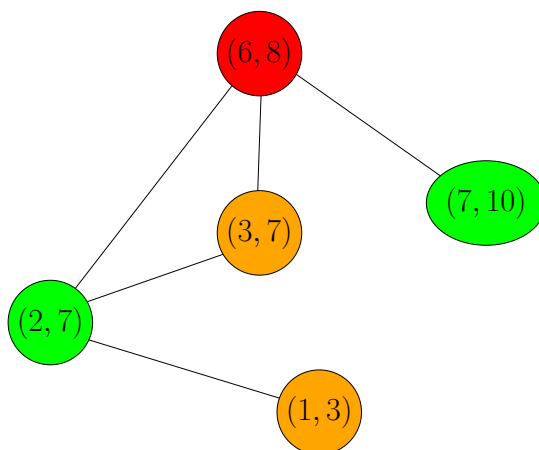
Dané hodiny tedy chceme rozdělit do co nejmenšího počtu učeben. Každou z učeben si můžeme označit nějakou barvou. Nyní sestojíme tzv. **intervalový graf**. Jeho vrcholy představují dané vyučovací hodiny, resp. příslušné intervaly, přičemž mezi dvojicí vrcholů vede hrana, právě když dané intervaly mají neprázdný průnik, tj. obecně $(a_i, b_i) \cap (a_j, b_j) \neq \emptyset$ (to znamená, že se hodiny svým časem kryjí). Zde si všimněme, proč se nám hodí pracovat s otevřenými intervaly. Pokud by dvojice hodin navazovala těsně po sobě ve stejné učebně, např. $(7, 9)$ a $(9, 10)$, pak bychom takovou situaci také mohli shledat přípustnou. Pokud bychom však používali uzavřené intervaly, pak by průnik vycházel neprázdný. Např. viz obrázek 6. Nyní každý vrchol obarvíme barvou nějaké učebny. Přitom dvojice vyučovacích hodin



Obrázek 6: Příklad intervalového grafu pro intervaly $(2, 7), (6, 8), (7, 10), (1, 3), (3, 7)$.

nesmí být rozvržena do stejné učebny, pokud se časově kryjí. To znamená, že sousední vrcholy musí mít vždy různé barvy (viz obrázek 7). Zde si všimněte, že libovolná skupina vrcholů obarvená stejnou barvou tvoří nezávislou množinu.

V praxi je (efektivní) rozvrhování běžnou záležitostí (např. rozvrhování procesů mezi vlákna). Později si však vysvětlíme, proč je tento problém v obecném případě algoritmicky těžký.



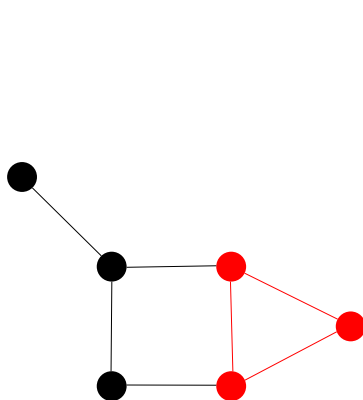
Obrázek 7: Obarvení intervalového grafu pro intervaly $(2, 7)$, $(6, 8)$, $(7, 10)$, $(1, 3)$, $(3, 7)$.

5 Problém kliky v grafu

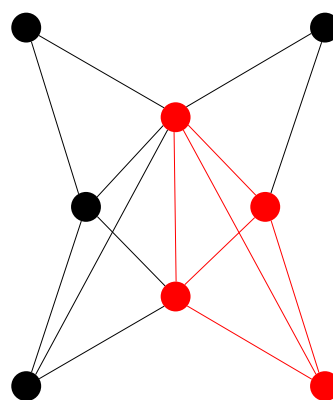
Jako poslední si ještě ukážeme ještě jeden typ problému, který (jak uvidíme) přímo souvisí s problémem nezávislé množiny. Nejdříve si však ještě připomeňme některé důležité termíny z teorie grafů. Jako první si definujeme tzv. *kliku*.

Definice 5.1 (Klika). Necht je dán graf $G = (V, E)$. Klikou nazveme libovolný podgraf $G' = (V', E')$, kde $V' \subseteq V$ a $E' \subseteq E$, který tvoří *úplný graf* libovolné velikosti.

Příklad 5.2. Příklady klik v grafech G_1 a G_2 , viz obrázky 8a a 8b.



(a) Klika v grafu G_1 .



(b) Klika v grafu G_2 .

Podobně jako u nezávislé množiny, i zde platí, že každý graf jistě obsahuje kliku (např. samostatný vrchol je z definice klika). Bude nás tedy opět zajímat, zda existuje v zadaném grafu klika určité velikosti.

PROBLÉM KLIKY V GRAFU (KLIKA)

Vstup problému: Graf $G = (V, E)$ a číslo $k \in \mathbb{N}$.

Výstup problému: 1, pokud v G existuje klika velikosti alespoň k , jinak 0.

Nyní si ukážeme, že ve skutečnosti problém kliky je ekvivalentní s problémem nezávislé množiny. K tomu si však definujeme ještě jeden termín.

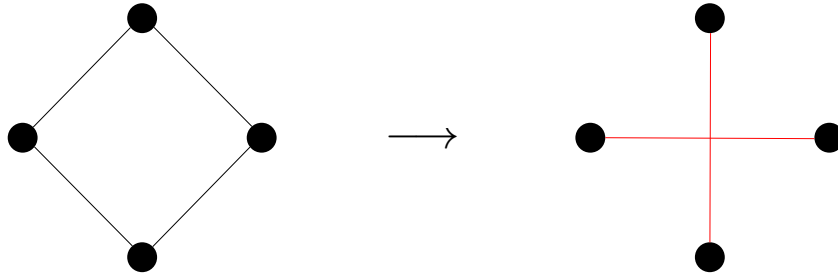
Definice 5.3 (Doplňek grafu). Je-li $G = (V, E)$ graf, pak doplňkem grafu G je graf $\overline{G} = (V, E')$, kde platí:

$$(u, v) \in E \iff (u, v) \notin E',$$

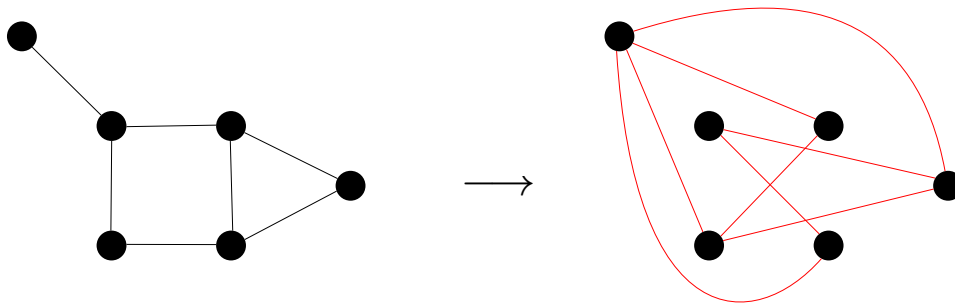
pro každou dvojici vrcholů u, v grafu G .

Co tato definice vlastně říká? Jednoduše to, že doplňek grafu G obsahuje právě ty hrany, které v původním grafu chybí. Podívejme se na příklady.

Příklad 5.4. Příklady doplňků grafů, viz obrázky 9 a 10.



Obrázek 9: Doplňek grafu na čtyřech vrcholech.



Obrázek 10: Doplňek prasátkového grafu.

5.1 Převod Klika na NzMna

Jako poslední se nyní zamyslíme, jaká je tedy souvislost mezi problémem nezávislé množiny a kliky v grafu. Předpokládejme, že máme tedy zadaný graf G a číslo $k \in \mathbb{N}$. Chceme sestavit graf G' a číslo k' takové, že v G existuje klika velikosti alespoň k právě, když novém grafu G' existuje nezávislá množina velikosti alespoň k' . Toto zařídíme však velice jednoduše.

Pro daný graf G sestavíme jeho doplňek $\overline{G}$. Nyní se zamysleme. Pokud v G existuje nějaká klika (pro tuto chvíli nezáleží na její velikosti), pak její vrcholy tvoří v $\overline{G}$ nezávislou množinu. Toto je jasné, neboť všechny vrcholy, které byly spojeny hranou v G nyní v $\overline{G}$ spojeny hranou nejsou. Tedy všechny dvojice vrcholů dané kliky nyní nejsou spojeny hranou.

Tedy celkově stačí položit $G' = \overline{G}$ a $k' = k$ a jsme hotovi. Opačný převod provedeme naprosto stejně.

6 Třídy problémů P a NP

V této sekci se nyní ohledneme trochu zpět a podíváme se na celou problematiku z více abstraktního hlediska. Naším cílem je představit si dvojici základních tříd problémů a vysvětlit si jejich podstatu v současném světě informatiky. Jako první se trochu blíže podíváme na problémy, které jsme si již představili.

6.1 Rozbor předchozích problémů

V předchozích sekcích jsme si představili některé základní problémy. Všimněte si, že vždy jsme se ptali na to, zda nějaký objekt splňuje určitou vlastnost. Např. u logické formule jsme se ptali, zdali je či není splnitelná. U grafů jsme se ptali, zda v existuje nezávislá množina či klika určité velikosti. Výstupem problému tedy byly vždy pouze hodnoty 1 (pravda) nebo 0 (nepravda). Takovým problémům se říká tzv. **rozhodovací problémy**.

Mimo tento typ se však v praxi spíše setkáme s tzv. **optimalizačními problémy**. V praxi nás nejspíše nebude zajímat, zda v grafu existuje klika určité velikosti. Spíše nás bude zajímat, jaká největší klika v zadaném grafu existuje. Ve skutečnosti optimalizační varianty problémů však umíme vždy řešit pomocí těch rozhodovacích. Kdybychom uměli rychle rozhodnout, zda existuje v grafu klika určité velikosti, pak bychom pomocí příslušného algoritmu uměli vyřešit i optimalizační verzi. Graf $G = (V, E)$ ponecháme stejný, akorát budeme měnit parametr k , tj. velikost množiny. Nejednodušší variantou je iterovat daný parametr k , dokud nám algoritmus nevydá jako odpověď *pravdu*, tj. 1.

Uvažujme, že podprogram `ExistsClique( $G, k$ )` určuje pro vstupní graf G a číslo k , zda v něm existuje klika velikosti alespoň k . Pomocí něj pak umíme vyřešit i optimalizační verzi, viz 6.1. Podobně tak lze

Algoritmus 6.1: Určení maximální kliky v grafu

Vstup: Graf $G = (V, E)$

```
1  $n \leftarrow |V|$ 
2 for  $k = n, n - 1, n - 2, \dots, 1, 0$  do
3   if ExistsClique( $G, k$ ) = 1 then return  $k$ ;
```

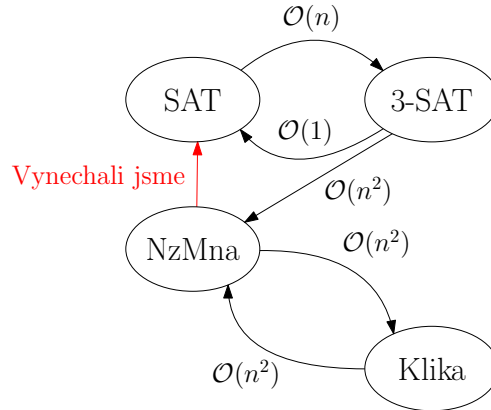
Výstup: Velikost maximální kliky v grafu G

řešit i ostatní problémy. Pro nás je však postatné si uvědomit, že naše omezení na rozhodovací problémy má tedy hned dva důvody. Celkově nám to zjednodušuje pohled na danou problematiku a navíc od rozhodovacího problému k optimalizačnímu umíme přejít velice snadno.

Vraťme se tedy zpět k rozhodovacím variantám, které jsme již probírali. Zkusme si rozebrat rychlosti jednotlivých převodů.

- $SAT \rightarrow 3\text{-}SAT$. Klauzuli s $k > 3$ literály rozložíme v čase $\mathcal{O}(k)$. Převod tedy potrvá maximálně lineárně dlouho v délce formule, tedy počtu jejích literálů n .
- $3\text{-}SAT \rightarrow SAT$. Formule v 3-CNF jsou již v CNF, tedy převodem nestrávíme žádný čas.
- $3\text{-}SAT \rightarrow NzMna$. Sestrojený graf má vrcholy v počtu literálů formule, to potrvá $\mathcal{O}(n)$. Hran v grafu bude maximálně kvadraticky mnoho (tj. úplný graf), tedy $\mathcal{O}(n^2)$. Celkově $\mathcal{O}(n + n^2) = \mathcal{O}(n^2)$.
- $Klika \rightarrow NzMna$. Doplněk grafu má vrcholy původního grafu. Hran sestrojíme maximálně kvadraticky mnoho, tj. celkově $\mathcal{O}(n^2)$.
- $NzMna \rightarrow Klika$. Stejně jako u předchozího převodu.

Můžeme si všimnout, že všechny převody jsou polynomiální, tedy jak jsme uváděli v úvodu kapitoly, můžeme je považovat za “rychlé”, viz obrázek 11. Co však dodnes neumíme je kterýkoliv z nich v polynomiálním čase řešit. Dané převody je nám však dávají hezky do souvislosti. Pokud bychom uměli řešit kterýkoliv z nich rychle, pak bychom uměli řešit rychle všechny z nich. Stačilo by nám libovolný jiný z nich převést na ten, který rychle řešit umíme. Např. pokud bychom byli schopni řešit problém kliky



Obrázek 11: Shrnutí převodů mezi jednotlivými problémy.

v polynomiálním čase, tj. $\mathcal{O}(n^\ell)$ pro nějaké ℓ , potom např. problém 3-SAT bychom mohli řešit tak, že jej v čase $\mathcal{O}(n^2)$ převedeme na problém nezávislé množiny, čímž dostaneme instanci velikosti m a tu následně převedeme opět v čase $\mathcal{O}(m^2)$. Celkově jsme instanci problému 3-SAT převedli v čase $\mathcal{O}(n^2 + m^2)$ na instanci problému kliky, kterou nyní vyřešíme v $\mathcal{O}((n^2 + m^2)^\ell)$. To je ovšem po roznásobení zase polynom.

6.2 Ověřování problémů

Už jsme si zmínili, že žádné ze zmíněných problémů (ani mnohé jiné) neumíme řešit v polynomiálním čase, ač instance mnohých z nich dnes umíme řešit v uspokojivých časech. U hodně z nich však umíme rychle ověřit, zda je nějaké řešení správné. Uvažujme situaci, kdy máme zadanou logickou formuli φ a nějaké ohodnocení jejích proměnných. Zajímá nás, zda je dané ohodnocení splňující. To není ale vůbec těžké. Zkrátka danou formuli vyhodnotíme. To však již umíme provést rychle!

- Pokud je zadané ohodnocení splňující, pak je φ splnitelná.
- Pokud ohodnocení není splňující, pak to ještě nutně *neznamená, že je nesplnitelná*.

Všimněme si tedy, že nyní se naše otázka změnila.

- Původní otázka: *Je zadaná formule φ splnitelná?*
- Současná otázka: *Je zadané ohodnocení proměnných zadané formule φ splňující?*

Pojďme se podívat touto optikou ještě na jiný problém. U problému NzMna nás zajímalo, zda zadaný graf G obsahuje nezávislou množinu velikosti alespoň k . Nyní nás bude zajímat následující otázka: *Je zadaná množina N v grafu G nezávislá velikosti alespoň k ?* Opět jsme přeformulovali otázku do podoby, kdy původně hledaný¹⁵ objekt již máme zadaný a pouze ověřujeme, zda splňuje danou vlastnost. Zde je řešení opět velice jednoduché, zkrátka zkontrolujeme, zda je množina dostatečně velká a zda mezi žádnou dvojicí vrcholů nevede hrana. Viz algoritmus 6.2.

¹⁵Formálně nelze přímo říci, že takový objekt vyloženě “hledáme”. Pouze nás zajímá, zda existuje.

Algoritmus 6.2: Ověření nezávislé množiny

```
Vstup: Graf  $G = (V, E)$ , číslo  $k \in \mathbb{N}$  a množina  $N \subseteq V$   
// Množina  $N$  není dostatečně velká  
1 if  $|V| < k$  then return 0;  
   // Dvojice vrcholů je spojena hranou  
2 foreach  $u \in V$  do  
3   foreach  $v \in V; v \neq u$  do  
4     if  $\{u, v\} \in E$  then return 0;  
   // Množina  $N$  je nezávislá  
5 return 1
```

Časová složitost je zjevně polynomiální. Označíme $|N| = m$, přičemž jistě platí $m \leq |V|$. Vnější cyklus se provede m -krát a vnitřní cyklus $(m-1)$ -krát (vrchol u jako jediný znovu nevybíráme). Tedy celkově

$$\mathcal{O}(m(m-1)) = \mathcal{O}(m^2 - m) = \mathcal{O}(m^2),$$

což je polynom. Tedy umíme rychle ověřit, zda je množina nezávislá.

Podobně si můžete rozmyslet, jak budou vypadat takové “ověřovací” algoritmy pro ostatní problémy. Co je ovšem podstatou tohoto výkladu? Obecně existují problémy, které jsme schopni řešit s určitou “náповědou”. Např. v případě nezávislé množiny si můžeme nechat napovědět nějakou podmnožinu vrcholů $N \subseteq V$ a nám jen stačí ověřit, zda je skutečně nezávislá a dostatečně velká. Podobně u problému SAT si necháme napovědět ohodnocení proměnných a nám pouze stačí ověřit, zda danou formuli splňuje.

Poznámka 6.1. Zde poznamenejme, že ne u každého problému jsme schopni rychle ověřit správnost řešení. Např. u Hanojských věží nejen že nejsme schopni rychle nalézt řešení, ale dokonce nejsme ani schopni jej rychle ověřit. Vždy budeme potřebovat vzhledem k počtu disků exponenciálně mnoho kroků, tedy i odsimulování poskytnutého řešení potrvá dlouho.

6.3 Zavedení tříd složitosti

Předchozí úvahy nám dávají rozdělení problémů do dvou základních kategorií.

- Ty, které *umíme řešit v polynomiálním čase*
- a ty, u nichž *dokážeme v polynomiálním čase ověřit správnost řešení*.

Tím dostáváme tzv. *třídy složitosti* P a NP (též se jim nazývá *třídy problémů*). Pojďme se na ně podívat.

P je třída rozhodovacích problémů řešitelných v polynomiálním čase.

Třída P tedy zachycuje všechny problémy, které jsou pro v jistém smyslu rychle řešitelné¹⁶.

Příklad 6.2. Příklady některých (rozhodovacích) problémů, které jsou řešitelné v polynomiálním čase.

¹⁶Asi by čtenáře mohlo napadnout, že např. algoritmus s časovou složitostí $\mathcal{O}(n^{1000})$ je asi jen stěží použitelný. Nebo podobně z druhé strany algoritmus s časovou složitostí $\mathcal{O}(1,001^n)$ je jistě prakticky použitelný pro určité velikosti dat, byť je exponenciální. Takové případy jsou však velice řídké a často platí, že u polynomiálního algoritmu s vysokou mocninou jsme často schopni ji snížit, neboť existuje efektivnější řešení. Proto se v tomto ohledu nekladou žádná další omezení, neboť to i při další práci podstatně zjednodušuje celou situaci.

- (i) *Existence cesty mezi dvěma vrcholy.* Lze řešit např. pomocí BFS, DFS¹⁷, Dijkstrova algoritmu nebo A^* , o nichž víme, že jsou všechny polynomiální (vyjma A^* , u nějž jsme si toto neukazovali).
- (ii) *Existence podřetězce v textu.* Máme zadaný textový řetězec délky n a máme zadané slovo (tj. posloupnost znaků) délky k . Zajímá nás, zda se dané slovo v textu nachází, či nikoliv. To můžeme zjistit např. testem všech řetězců délky k . Těch bude $\mathcal{O}(n - k)$.
- (iii) *Existence prvku v poli.* Lze řešit např. sekvenčním vyhledáváním, které běží v čase $\mathcal{O}(n)$.

Všechny zmíněné případy problémů jsou řešitelné v čase $\mathcal{O}(n^k)$.

NP je třída rozhodovacích problémů, u nichž lze v polynomiálním čase ověřit jejich řešení.



Třídy P a NP zcela úmyslně neuvádíme jako definice a je potřeba je chápat s jistou rezervou. Nejedná se totiž o správný a přesný výklad dané problematiky, avšak pro naše potřeby si s těmito “definicemi” vystačíme.¹⁸

Problémy třídy NP jsou již trochu zajímavější, neboť již nutně nepožadujeme, aby byl problém rychle řešitelný. Podmínkou je pouze, že musíme být schopni rychle ověřit, zda zadaný objekt splňuje požadovanou vlastnost. Alternativně též můžeme na třídu NP nahlížet tak, že daný problém jsme schopni řešit rychle s určitou nápovědou.

Příklad 6.3. Příklady problémů ležících ve třídě NP jsme si již ukazovali v sekci 6.1, ale ještě jednou si je zde shrneme.

- (i) *SAT, 3-SAT.* Jako nápověda zde slouží ohodnocení proměnných. Stačí ověřit, zda je splňující vyhodnocením zadané formule.
- (ii) *Nezávislá množina.* Na vstupu dostaneme kromě grafu G a čísla k ještě nějakou množinu vrcholů N . Nám posléze stačí rozhodnout, zda N je nezávislá a platí $|N| \geq k$. Viz algoritmus 6.2.
- (iii) *Klika.* Opět dostaneme množinu vrcholů N . Postup podobný jako u NzMna.

Může se na první pohled zdát, že třídy P a NP představují dva oddělené světy. Ve skutečnosti tomu tak není. Lze si totiž všimnout zajímavého vztahu, který má kupodivu jednoduché zdůvodnění.

Věta 6.4. Platí $P \subseteq NP$.

Věta 6.4 říká, že třída P leží uvnitř třídy NP, viz obrázek 12. Pokud totiž dokážeme nějaký problém $A \in P$ řešit v polynomiálním čase (tedy bez nápovědy), tím spíš jej dokážeme řešit rychle s nápovědou (např. algoritmus může nápovědu ignorovat a určit řešení přímo z původního vstupu). Tento fakt ovšem zahrnuje i potenciální variantu, že $P = NP$. To ovšem dodnes není známo a otázka, zda jsou třídy P a NP skutečně různé zůstává stále nezodpovězena¹⁹. Obecně ale spíše přijímána hypotéza, že jsou tyto třídy různé.

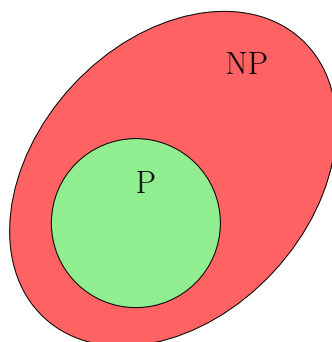
U všech zmíněných problémů v této kapitole s jistotou víme, že leží ve třídě NP, avšak nevíme, zda leží i ve třídě P, neboť pro žádný z nich neznáme polynomiální algoritmus. Avšak lze o nich ukázat, že jsou tyto problémy (a mnohé jiné) v jistém smyslu “ty nejtěžší” ve třídě NP.

Definice 6.5 (NP-úplný problém). Problém A nazveme NP-úplným, pokud $A \in NP$ a zároveň je na něj polynomiálně převoditelný každý problém z NP. Tzn.

$$\forall L \in NP : L \rightarrow A.$$

¹⁷O DFS již víme, že nenalezne nutně nejkratší cestu, ale zde řešíme pouze její existenci.

¹⁹Problém P vs. NP patří mezi tzv. *Problémy tisíciletí*, které vyhlásil Clayův matematický institut v roce 2000. Na vyřešení každého z nich vypsál institut jeden milión amerických dolarů.



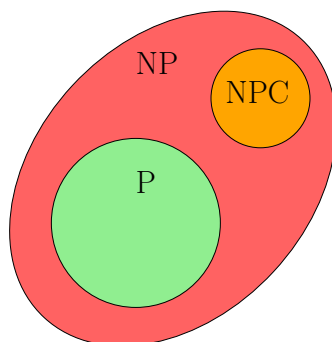
Obrázek 12: Ilustrace vztahu P a NP.

Druhá podmínka v definici 6.5 se možná zdá hodně silná. Existuje vůbec takový problém? Ač se to může zdát nepravděpodobné, tak v roce 1971 dokázal americký informatik Stephen Cook překvapivou větu.

Věta 6.6. Problém SAT je NP-úplný.

Důkaz toho tvrzení si zde ukazovat nebudeme, neboť je zcela nad rámec tohoto výkladu. Tento výsledek má v této teorii velice silný důsledek. Pokud by se ukázalo, že nějaký NP-úplný problém A leží ve třídě P , tak by z toho již plynulo, že $P = NP$. Každý problém z NP bychom mohli jednoduše polynomiálně převést na A a ten pak vyřešit opět polynomiálně. Toto je přesně důvod, proč problém SAT hraje v současné informatice takovou důležitou roli, neboť se historicky ukázalo, že pomocí něj lze řešit kterýkoliv jiný problém²⁰ v NP .

NP-plných problémů existuje více. Třída takových problémů se označuje jako NPC (z anglického *NP-Complete*) a též platí $NPC \subseteq NP$, viz obrázek 13. Pokud jsme schopni nějaký NP-úplný problém A převést na jiný problém B , pak i problém B je nutně NP-úplný. Z toho plyne, že i problémy *SAT*, *3-SAT*, *NzMna* a *Klika* jsou též NP-úplné.



Obrázek 13: Ilustrace vztahu P, NP a NPC.

Jak už jsme si ale uvedli, to zda platí $P = NP$, stále nevíme. Jaké by to ale mělo důsledky v praxi? Pojďme se alespoň na chvíli zamyslet nad oběma variantami.

- $P = NP$. V tomto případě by každý vyhledávací problém byl rychle řešitelný, tedy nejpíše i prakticky a efektivně. Např. uvedený problém s rozvrhováním (viz podsekcce 4.2) by byl rychle řešitelný. Zároveň bychom tím ale přišli i o veškeré efektivní šifrování, které je z velké části přímo založené na hypotéze, neboť ke každé šifrovací funkci bychom mohli rychle spočítat i její inverzi.

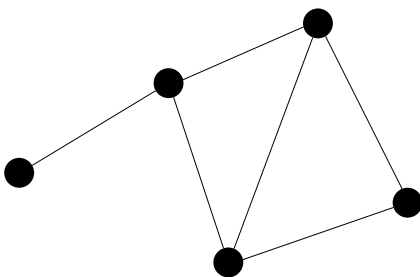
²⁰Proto se někdy též říká, že NP-úplné problémy tvoří v jistém smyslu ty “nejtěžší” v NP , neboť pokud umíme řešit nějaký z nich, pak umíme vyřešit všechny.

- $P \neq NP$. V tomto světě jsou třídy problémů P a NP -úplných problémů různé. SAT, NzMna, Klika a mnohé jiné problémy nejsme schopni řešit rychle bez nápovědy. Naopak by byly “posíleny” mnohé kryptografické algoritmy.

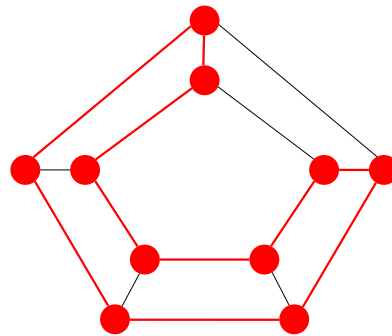
6.4 Další NP-úplné problémy

K závěru této kapitoly si ještě uvedeme nějaké další NP-úplné problémy a k nim i nějaké příklady jejich aplikací (resp. výskytů). NP-úplnost si zde však již dokazovat pro žádný z nich nebudeme. Pro žádný z těchto problémů není známý polynomiální algoritmus.

- (i) *Hamiltonovská kružnice*. Máme zadaný graf $G = (V, E)$. Zajímá nás, zda existuje v grafu G cyklus, který prochází všemi vrcholy. Viz obrázky 14a a 14b. Problém hledání hamiltonovské kružnice



(a) Příklad grafu, kde neexistuje hamiltonovská kružnice



(b) Graf a jeho hamiltonovská kružnice

se objevuje typicky např. v logistice, kdy se snažíme optimalizovat trasy při doručování zboží (minimalizace času a nákladů).

- (ii) *Součet podmnožiny*. Máme zadanou množinu přirozených čísel M a číslo $S \in \mathbb{N}$. Existuje podmnožina $N \subseteq M$ taková, že součet jejích čísel je roven S ? Např. pro množinu $M = \{1, 3, 10, 11, 20\}$ (ne)existují součty uvedené v tabulce 1. Rychlé řešení součtu podmnožiny by pomohlo např. s op-

Součet S	13	2	22	16
Existence	✓	✗	✓	✗

Tabulka 1: Tabulka součtů množiny M

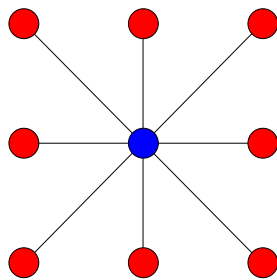
timalizací plánování nákladů rozpočtem (např. ve firmě). Dále se též používá u tzv. *jednosměrných funkcí* v kryptografii.

- (iii) *Barvení grafu*. Chceme zjistit, zda pro zadaný graf je možné jeho vrcholy obarvit k barvami (formálně přidělíme každému vrcholu číslo od 1 do k) tak, aby vrcholy stejné barvy nebyly nikdy spojeny hranou? Pro příklady viz obrázky 15a a 15b.
- (iv) *Problém batohu*. Jsou dány předměty s váhami a cenami a kapacita batohu. Chceme najít co nejdražší podmnožinu předmětů tak, abychom nepřekročili kapacitu batohu. Toto je nejčastější zadání, avšak pokud bychom chtěli problém upravit na rozhodovací, tak se budeme ptát, zda existuje podmnožina předmětů s celkovou cenou větší nebo rovnou zadané ceně C .

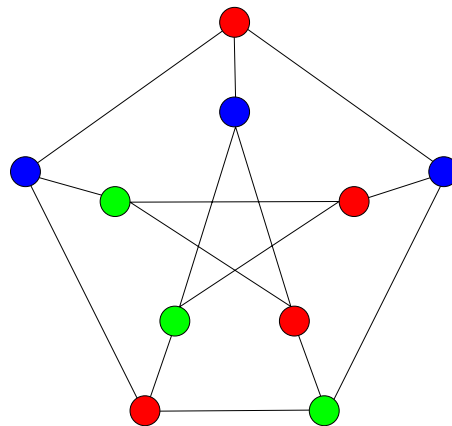
Formálněji máme váhy předmětů $w_1, \dots, w_n$ a jejich ceny $c_1, \dots, c_n$, přičemž hledáme hodnoty $x_1, \dots, x_n \in \{0, 1\}$ tak, aby platilo

$$\sum_{i=1}^n c_i x_i \geq C \quad \text{a} \quad \sum_{i=1}^n w_i x_i \leq W.$$

Problém batohu se též objevuje v kryptografii.



(a) Barvení grafu pro $k = 2$.



(b) Barvení grafu pro $k = 3$.

- (v) *Dva loupežníci*. Ptáme se, zda lze zadanou množinu čísel rozdělit na dvě podmnožiny se stejným součtem²¹. Např. $L = \{1, 3, -2, -1, -16\}$ lze rozdělit na množiny $L_1 = \{1, 3, -16\}$ a $L_2 = \{-2, -10\}$. S tímto problémem se lze setkat např. u distribuovaných systémů, kdy se snažíme spravedlivě rozdělit omezené výpočetní zdroje mezi dva nezávislé procesy.

²¹Loupežníci tedy vlastně řeší, jak spravedlivě rozdělit lup.